Prof. Ashwani Mishra
Assistant Professor
CSE Department

1. Introduction to NoSQL Databases

What is NoSQL?

NoSQL (Not Only SQL) databases are databases that do not use traditional relational tables like MySQL or Oracle.

Instead they store data in:

- Documents
- Key-value pairs
- Graphs
- Columns

NoSQL databases are designed for:

- Big Data
- High performance
- Flexible schema
- Cloud applications

Types of NoSQL Databases

Type	Example	Data Format
Document Database	MongoDB	JSON
Key Value	Redis	key:value
Column Based	Cassandra	Columns
Graph	Neo4j	Nodes

SQL vs NoSQL

SQL	NoSQL

Table based	Document based
Fixed schema	Dynamic schema
Vertical scaling	Horizontal scaling
Example: MySQL	Example: MongoDB

Example

Student record.

SQL Database

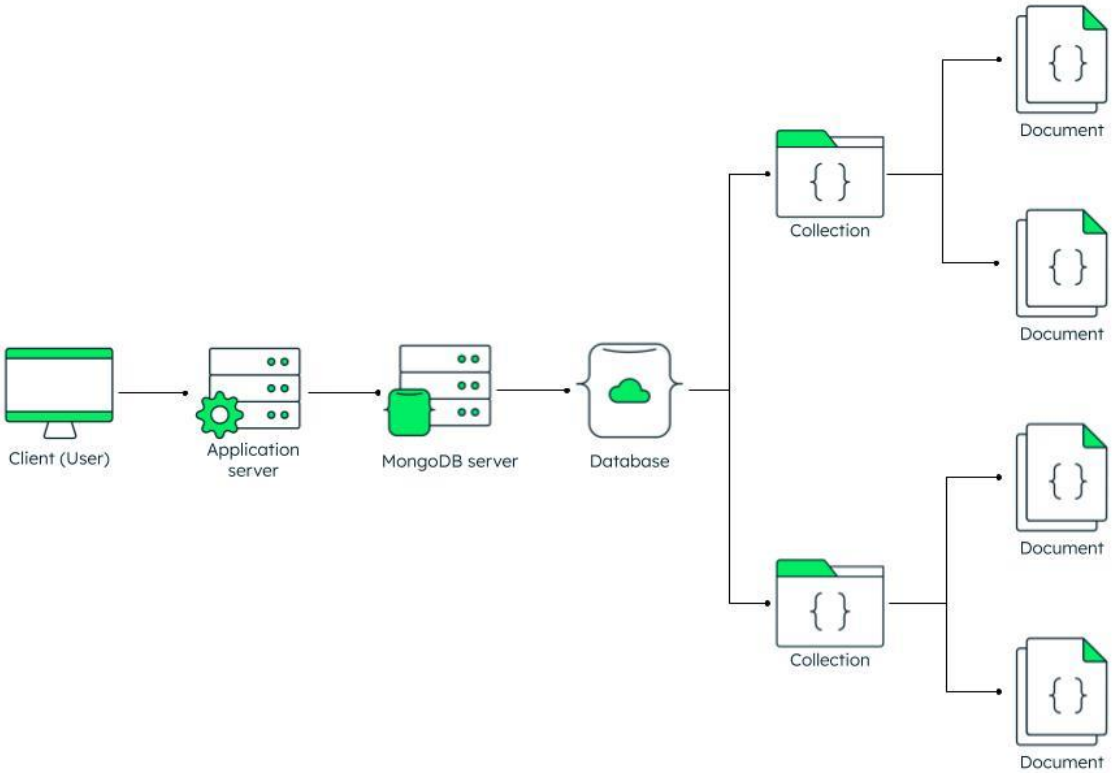
Id	**Name**	**course**
1	Rahul	CSE

MongoDB:

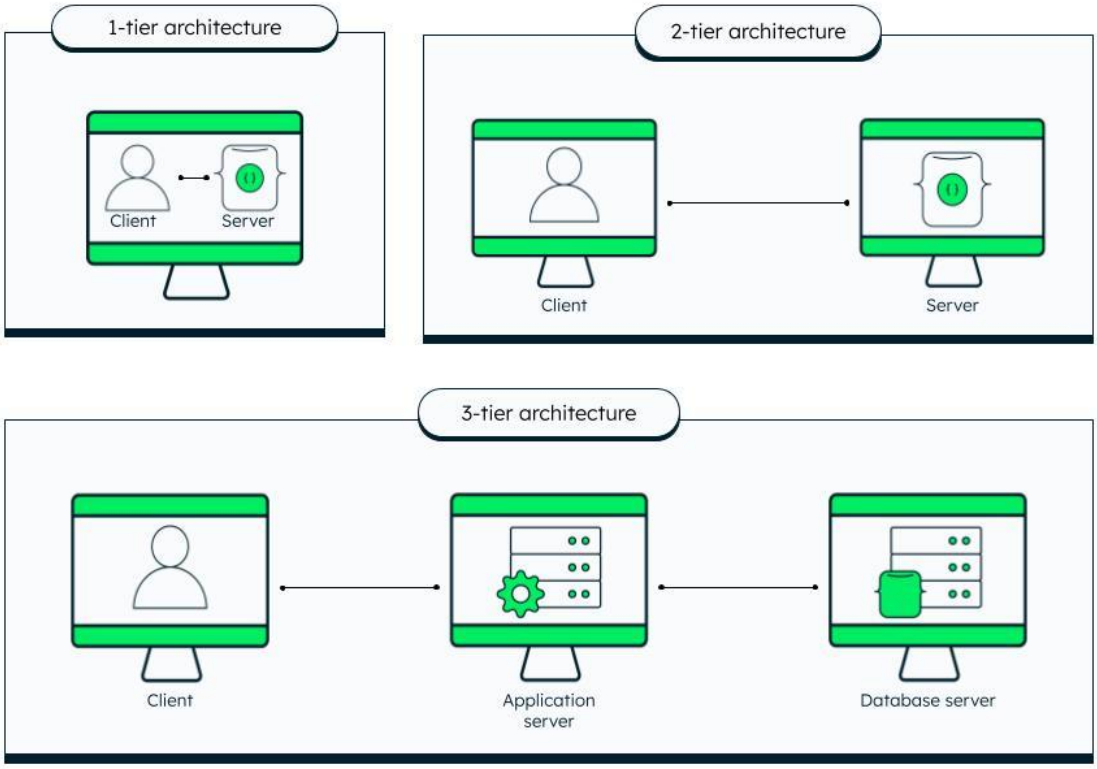
```
{
 "id":1,
 "name":"Rahul",
 "course":"BCA"
}
```

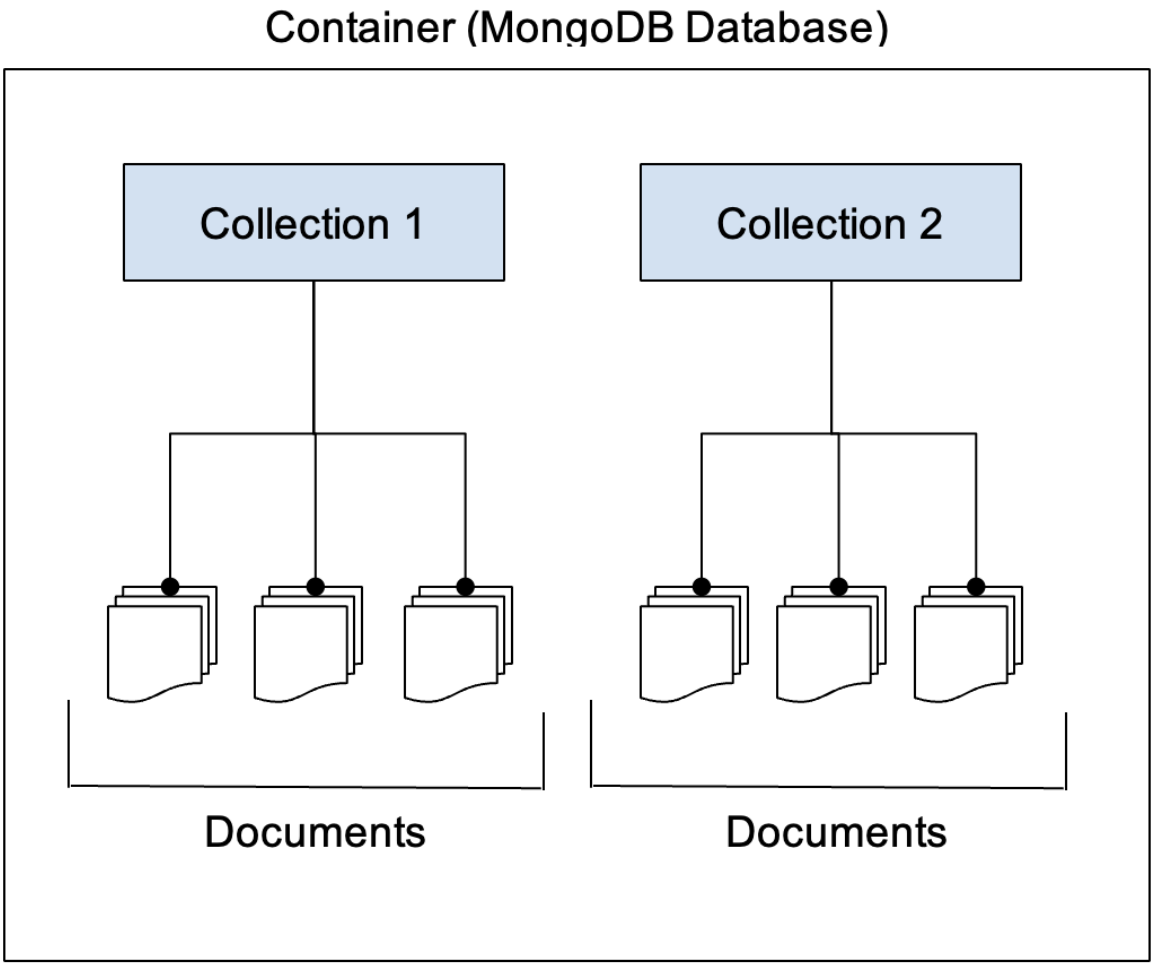
MongoDB Architecture

MongoDB basic architecture



Client server architecture





MongoDB Structure

Database

↓

Collection

↓

Document

↓

Fields

Example

CollegeDB

↓

Students

↓

```
{
 name:"Rahul",
 age:20
}
```

2. Installation and Configuration of MongoDB

Step 1 Download MongoDB

Download from:

https://www.mongodb.com/try/download/community

Step 2 Install MongoDB

Install using default settings.

MongoDB will install:

- MongoDB Server
- MongoDB Compass (GUI tool)

Step 3 Start MongoDB Server

Command:

Mongod

Step 4 Start Mongo Shell

Open another terminal:

Mongo

- Now MongoDB shell is ready.

Check Databases

show dbs

Example Output:

admin

local

config

3. CRUD Operations in MongoDB

CRUD means:

Operation	Meaning
C	Create
R	Read
U	Update
D	Delete

Create Database

MongoDB creates database automatically.

- use collegeDB

Create Collection

- db.createCollection("students")

Check collections

- show collections

Insert Data (CREATE)

- **Insert One Document**

```
db.students.insertOne({
 name:"Rahul",
 age:20,
 course:"BCA"
})
```

- **Insert Multiple Documents**

```
db.students.insertMany([
 {name:"Amit", age:22, course:"BBA"},
 {name:"Riya", age:21, course:"MCA"}
])
```

Read Data (SELECT)

- Show All Data

  ```
  db.students.find()
  ```

- Pretty Format

  ```
  db.students.find().pretty()
  ```

- Find Specific Data

  ```
  db.students.find({name:"Rahul"})
  ```

Update Data

Update One Document:

```
 db.students.updateOne(
 {name:"Rahul"},
 {$set:{age:23}}
)
```

Update Many Documents:

```
db.students.updateMany(
 {course:"BCA"},
 {$set:{course:"BCA Updated"}}
)
```

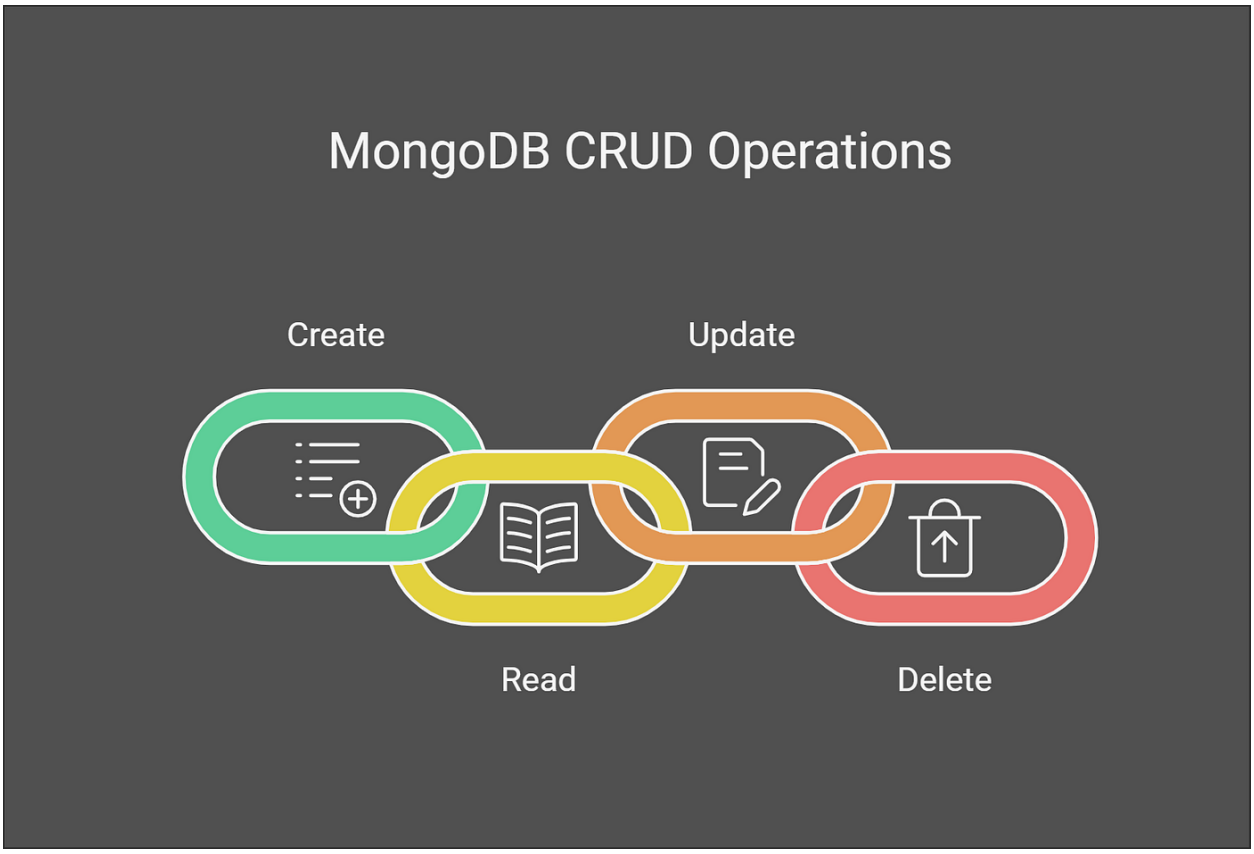
Delete Data:

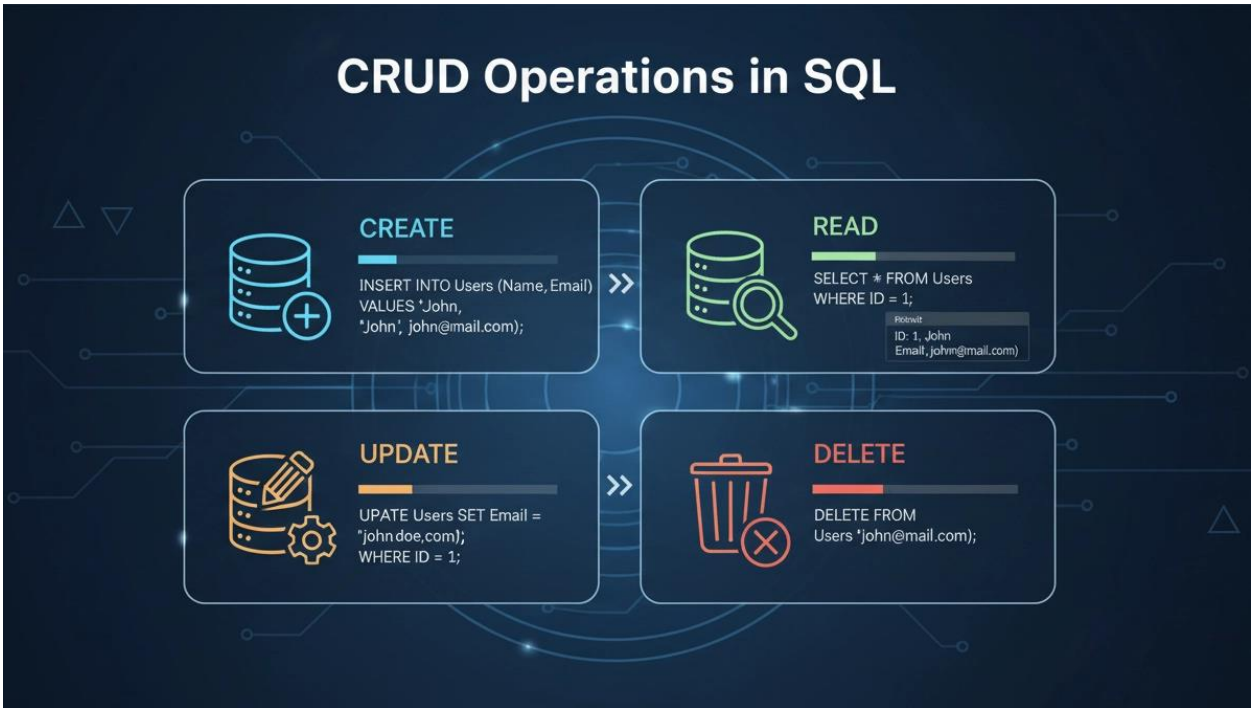
```
db.students.deleteOne({name:"Rahul"})
```

Delete Many Documents

```
db.students.deleteMany({course:"BBA"})
```

CRUD Flow Diagram





4. Indexing in MongoDB

What is Indexing?

Indexing improves database search speed.

Without index:

- Database checks every record.

With index:

- Database directly finds data.

Example Without Index

Search student name.

- 10000 records

Database checks each record, Slow.

Create Index

```
db.students.createIndex({name:1})
```

- 1 means **ascending order**

Descending Index

```
db.students.createIndex({age:-1})
```

Check Indexes

```
db.students.getIndexes()
```

5. Querying in MongoDB

Query means **searching data from database**.

Find Specific Field

```
db.students.find(
 {},
 {name:1, course:1}
)
```

Greater Than Query

```
db.students.find({age:{$gt:20}})
```

Less Than Query

```
db.students.find({age:{$lt:25}})
```

AND Query

```
db.students.find({
```

```
 age:{$gt:20},
 course:"BCA"
})
```

OR Query

```
db.students.find({
 $or:[
  {course:"BCA"},
  {course:"MCA"}
 ]
})
```

Query Operator Table

Operator	Meaning
$gt	Greater than
$lt	Less than
$gte	Greater than equal
$lte	Less than equal
$or	OR condition

6. Schema Design and Data Modeling

What is Schema?

Schema means structure of database data.

In MongoDB schema is flexible.

Example:

```
{
 name:"Rahul",
```

```
 age:21
}
```

Another document can have extra fields.

```
{
 name:"Amit",
 age:22,
 city:"Delhi"
}
```

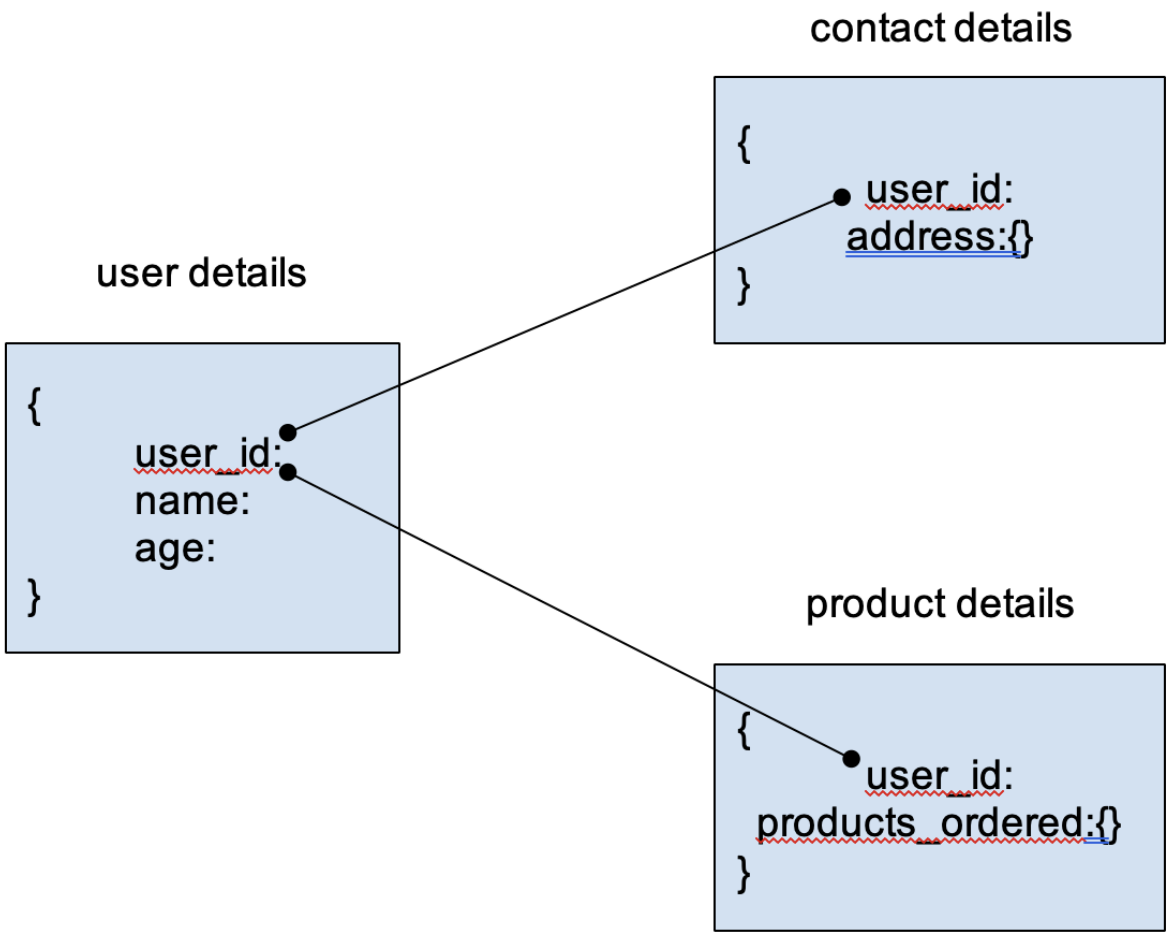
Data Modeling

Designing how data will be stored**.**

Two types:

1. Embedded Documents
2. Referenced Documents

Embedded Model



Example:

```
{
 name:"Rahul",
 course:"BCA",
 address:{
   city:"Lucknow",
   state:"UP"
 }
}
```

All data stored inside one document.

Advantages:

- Fast
- Simple queries

Reference Model

Example:

Students collection

```
{
 student_id:1,
 name:"Rahul"
}
```

Course collection

```
{
 course_id:101,
 course_name:"BCA",
 student_id:1
}
```

Here documents reference each other.

<u>Example Node.js + MongoDB Code:</u>

Install MongoDB driver

```
npm install mongodb
```

Example code:

```
const {MongoClient} = require("mongodb")

const url = "mongodb://localhost:27017"

MongoClient.connect(url)
.then(client=>{
```

```
const db = client.db("collegeDB")
const collection = db.collection("students")

collection.insertOne({
  name:"Rahul",
  age:21
})

console.log("Data Inserted")

})
```